NAME:SHIVANI

REG.NO.25MIM10234

COURSE:PYTHON ESSENTIALS

ACADEMIC YEAR: 2026

Project Title: Personal Expense Tracker with Smart Analysis

1. Introduction

- This project is a Python-based system designed to help users track and manage their daily expenses efficiently.
- It focuses on solving the common problem of unorganized financial records among students and individuals.
- The system allows users to record expenses in a structured format with details such as name, amount, and category.
- It provides a clear overview of spending patterns, which helps in better financial planning and control.
- The project is simple to use, does not require any technical background, and can run on any basic system.
- It demonstrates the practical application of Python programming concepts in solving real-life problems.

2. Problem Statement

- Many individuals fail to maintain a proper record of their daily expenses, leading to confusion and financial mismanagement.
- Small daily expenses are often ignored, but they accumulate into a large amount over time.
- Without categorization, it becomes difficult to analyze where most of the money is being spent.

- Lack of awareness about total expenditure results in overspending and reduced savings.
- There is a need for a simple, efficient, and user-friendly system to track and analyze expenses systematically.

3. Functional Requirements

- The system allows users to add new expenses by entering relevant details such as name, amount, and category.
- It provides an option to view all recorded expenses in a well-organized format.
- Users can calculate the total amount spent automatically without manual calculation.
- The system categorizes expenses into groups like food, travel, study, and others for better understanding.
- It includes functionality to delete specific expense records when needed.
- The system continuously interacts with the user through a menu-driven interface.

4. Non-Functional Requirements

- The system should be easy to use and understandable even for beginners.
- It must provide fast and efficient performance without delays.
- The program should be lightweight and should not require high system resources.
- It must handle invalid inputs properly to avoid crashes.
- The code should be structured and maintainable for future improvements.

5. Simple Flow of the System

- The user starts the program, and a menu with different options is displayed on the screen.
- The user selects an option such as adding, viewing, or deleting expenses.
- The system processes the user input and performs the requested operation.
- The result is displayed to the user in a clear and readable format.
- The process repeats until the user chooses to exit the program.

6. UML Diagrams

Use Case Diagram

```
    [User]
      |
 ---------------------
 |  |   |    |
 Add  View  Delete  Analyze
Expense Expense Expense Spending
```

Sequence Diagram

User → Menu: Select Option

Menu → System: Process Request

System → Data: Store/Retrieve

Data → System: Return Data

System → User: Display Output

Class Diagram

```
+----------------------+
|     Expense        |
+----------------------+
| name               |
| amount             |
| category           |
+----------------------+
| addExpense()       |
| viewExpense()      |
| deleteExpense()    |
| calculateTotal()   |
+----------------------+
```

7. Design Decision Rationale

- Python is selected due to its simplicity, readability, and beginner-friendly nature.
- Lists are used to store multiple expense records dynamically.
- Dictionaries are used to store structured data for each expense.
- A command-line interface is chosen to keep the system simple and lightweight.
- Functions are used to divide the program into manageable parts for better clarity and maintenance.

8. Implementation Details

- The system is implemented using Python programming language.
- A list is used as the main data structure to store all expense records.

- Each expense is stored as a dictionary containing name, amount, and category.
- Separate functions are defined for each operation such as adding, viewing, deleting, and analyzing expenses.
- A loop is used to repeatedly display the menu and allow continuous user interaction.
- Input validation is implemented to handle incorrect or unexpected inputs.

9. Code Testing Approach

- The system is tested using different input scenarios to ensure proper functionality.
- Both valid and invalid inputs are used to check error handling.
- Edge cases such as empty expense list are tested.
- Total and category calculations are verified for accuracy.
- Deletion functionality is tested to ensure correct removal of records.

10. Challenges Faced

- Handling incorrect user inputs such as invalid numbers or empty values.
- Designing a data structure that is simple yet efficient for storing records.
- Implementing category-wise calculations without errors.
- Ensuring the program runs smoothly without crashes.

11. Learning & Key Takeaways

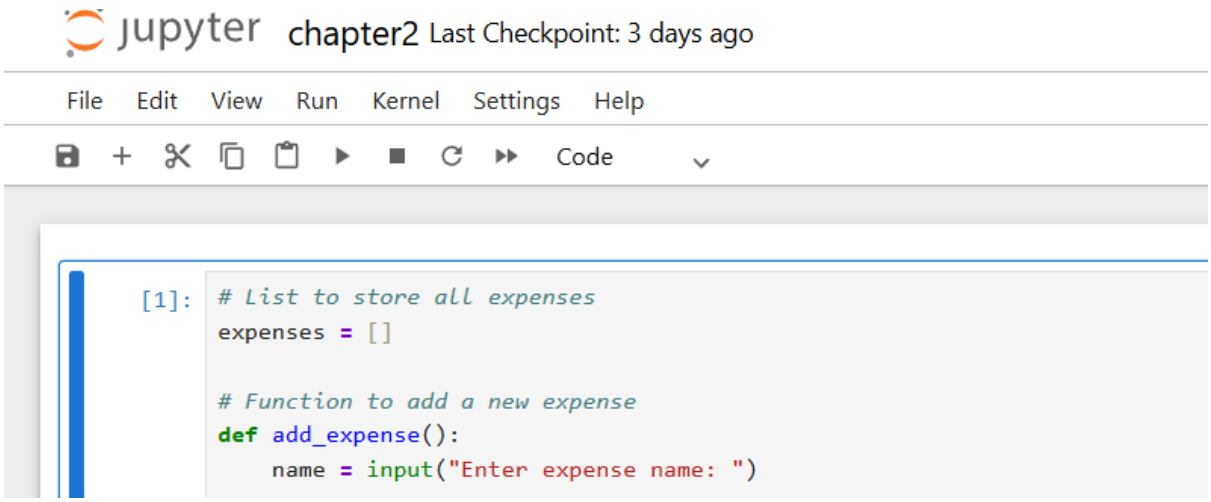
- Improved understanding of Python data structures like lists and dictionaries.

- Learned how to break a program into smaller functions for better structure.
- Gained experience in handling real-world problems using programming.
- Developed logical thinking and debugging skills.

12. Future Enhancements

- Add a graphical user interface (GUI) for better user experience.
- Store data permanently using files or databases.
- Add graphical charts for expense visualization.
- Implement budget alerts and notifications.

Code:



jupyter chapter2 Last Checkpoint: 3 days ago

File Edit View Run Kernel Settings Help

Code

```python
    # Input validation for amount
    while True:
        try:
            amount = float(input("Enter amount (₹): "))
            break
        except ValueError:
            print("Invalid amount! Please enter a number.")

    category = input("Enter category (Food/Travel/Study/Others): ")

    expense = {
        "name": name,
        "amount": amount,
        "category": category
    }

    expenses.append(expense)
    print("Expense added successfully!")

# Function to view all expenses
def view_expenses():
    if not expenses:
        print("No expenses recorded.")
    else:
        print("\nList of Expenses:")
        for i, exp in enumerate(expenses, start=1):
            print(f"{i}. {exp['name']} | ₹{exp['amount']} | {exp['category']}")

# Function to calculate total expenses
def total_expense():
    total = sum(exp["amount"] for exp in expenses)
    print(f"Total Expenses: ₹{total}")
```

jupyter chapter2 Last Checkpoint: 3 days ago

File Edit View Run Kernel Settings Help

Code

```python
# Function to show category-wise summary
def category_summary():
    if not expenses:
        print("No expenses recorded.")
        return

    summary = {}
    for exp in expenses:
        cat = exp["category"]
        summary[cat] = summary.get(cat, 0) + exp["amount"]

    print("\nCategory-wise Summary:")
    for cat, amt in summary.items():
        print(f"{cat}: ₹{amt}")

# Function to delete an expense
def delete_expense():
    view_expenses()
    if not expenses:
        return

    while True:
        try:
            index = int(input("Enter expense number to delete: ")) - 1
            if 0 <= index < len(expenses):
                removed = expenses.pop(index)
                print(f"Expense '{removed['name']}' deleted successfully!")
                break
            else:
                print("Invalid choice! Please enter a valid number.")
        except ValueError:
            print("Invalid input! Please enter a number.")
```

File Edit View Run Kernel Settings Help

Code

```python
            print("Invalid input! Please enter a number.")

# Main function with menu loop
def main():
    while True:
        print("\n--- Personal Expense Tracker ---")
        print("1. Add Expense")
        print("2. View Expenses")
        print("3. Total Expenses")
        print("4. Category Summary")
        print("5. Delete Expense")
        print("6. Exit")

        choice = input("Enter your choice: ")

        if choice == '1':
            add_expense()
        elif choice == '2':
            view_expenses()
        elif choice == '3':
            total_expense()
        elif choice == '4':
            category_summary()
        elif choice == '5':
            delete_expense()
        elif choice == '6':
            print("Exiting... Thank you!")
            break
        else:
            print("Invalid input! Please select a valid option.")

# Run the program
if __name__ == "__main__":
    main()
```

Output:

```
main()
```

```
--- Personal Expense Tracker ---
1. Add Expense
2. View Expenses
3. Total Expenses
4. Category Summary
5. Delete Expense
6. Exit
Enter your choice:  1
Enter expense name:  shivani
Enter amount (₹):  100
Enter category (Food/Travel/Study/Others):  study
Expense added successfully!

--- Personal Expense Tracker ---
1. Add Expense
2. View Expenses
3. Total Expenses
4. Category Summary
5. Delete Expense
6. Exit
Enter your choice:  2

List of Expenses:
1. shivani | ₹100.0 | study

--- Personal Expense Tracker ---
1. Add Expense
2. View Expenses
3. Total Expenses
4. Category Summary
5. Delete Expense
6. Exit
Enter your choice:  3
Total Expenses: ₹100.0
```

```
--- Personal Expense Tracker ---
1. Add Expense
2. View Expenses
3. Total Expenses
4. Category Summary
5. Delete Expense
6. Exit
Enter your choice:  4

Category-wise Summary:
study: ₹100.0

--- Personal Expense Tracker ---
1. Add Expense
2. View Expenses
3. Total Expenses
4. Category Summary
5. Delete Expense
6. Exit
Enter your choice:  5

List of Expenses:
1. shivani | ₹100.0 | study
Enter expense number to delete:  6
Invalid choice! Please enter a valid number.
Enter expense number to delete:  5
Invalid choice! Please enter a valid number.
Enter expense number to delete:  1
Expense 'shivani' deleted successfully!

--- Personal Expense Tracker ---
1. Add Expense
2. View Expenses
3. Total Expenses
4. Category Summary
5. Delete Expense
6. Exit
```

```
6. Exit
Enter your choice:  6
Exiting... Thank you!
```